# Assignment Cover Sheet

**This cover sheet must be completed and added to the front of every assignment**

| | |
|---|---|
| ***Learner Name and Surname:*** | Samsuddoha Sifat |
| ***Learner Registration No.*** | 13117 |
| ***Study Centre Name*** | **Learn Key** |
| ***Qualification Title*** | Undergraduate Diploma in Software Design LKI/MQF (Lv.5) (A) |
| ***Unit Reference No.*** | ICT-UDSDU9 |
| ***Unit Title*** | Award in Introduction to DevOps: Principles and Practices |
| ***Submission Date*** | 29th August, 2026 |

**Declaration of authenticity:**

1. **I declare that the attached submission is my own original work. No significant part of it has been submitted for any other assignment and I have acknowledged in my notes and bibliography all written and electronic sources used.**
2. **I acknowledge that my assignment will be subject to electronic scrutiny for academic honesty.**
3. **I understand that failure to meet these guidelines may instigate the centre's malpractice procedures and risk failure of the unit and / or qualification.**

| | |
|---|---|
| Samsuddoha Sifat<br>***Learner signature***<br>***Date:*** | ***Tutor signature***<br>***Date:*** |

Note:

Assignments must be submitted in typed PDF format only; handwritten assignments are not accepted.

# Executive Summary

This document outlines a delivery strategy for DevOps for TaskNest Ltd, which is a rapidly growing SaaS startup and has a web-based project management tool. Company has undergone considerable growth without going through automated deployment practices or manual configuration of servers. Therefore TaskNest Ltd finds itself in numerous operational troubles, such as multiple environmental discrepancies, reliance on one senior engineer having knowledge of all production processes, constant incidents and the increasing costs of cloud computing.

In the first section of the report, the current situation of this company is analyzed through several key principles of DevOps. In particular, the CALMS model is used to evaluate the Culture, Automation, Lean, Measurement, and Sharing aspects, while the Three Ways framework is implemented in order to analyze the Flow, Feedback and Continual Learning. Moreover, Lean waste has been established within the procedure of delivery of the software being utilized, while four DORA metrics have been used to identify the baseline of delivery performance.

The following five sections detail a plan for addressing issues using the DevOps methodology. Section two outlines version control and trunk-based development using Git and GitHub. Section three outlines a CI/CD process using automated testing and quality gates. Section four presents the creation of repeatable environments through Infrastructure as Code using Terraform, containerisation with Docker, and orchestration with Kubernetes. Section five discusses observability, DevSecOps, reliability, incident management, and disaster recovery. Section six concludes the plan with FinOps practices and an implementation roadmap covering six months.

Supporting documents such as Git history and GitHub Actions are attached as evidence of this plan.

# Part 1 - Current-State Analysis & the Case for DevOps

## 1.1 The situation at TaskNest

TaskNest is a cloud-based software-as-a-service solution, which had about 15 engineers developing its main product. After becoming too large for its own solution, the advancement has surpassed its deployment process, which is carried out entirely manually. When one of the developers creates a new feature, they rely on one engineer, Priya, to activate it in production. She then accesses the production server, logs in via SSH, and activates the release, hoping that it works smoothly. There is no staging environment that is similar to production and no tests being run.

The major signs of the problem include: (1) the environment drift  servers were manually set up for two years and no one knows what has been running on which server, (2) the bus factor is one  Priya is the only one aware of how the production works, (3) regular incident occurrences small changes lead to outages frequently and (4) the cloud bill is ignored old test servers are still up and running and the instances are oversized and are not tagged.

## 1.2 Analysis using the CALMS model

When applying the CALMS model (Kim et al., 2016), one can argue that TaskNest shows substandard performance on almost every level. Culture: 'Priya will handle it' creates a culture of knowledge hoarding since everyone relies on Priya's expertise rather than sharing information and learning systematically from mistakes; the blame for problems falls on individuals instead of being systematically analyzed. Automation: TaskNest does not use any technology for automation, and all processes are performed manually. Lean: there is a significant amount of waste and work is done in

large batches. Measurement: lead times and failures are not measured, resulting in no quantitative results. Sharing there are no documents explaining how the system works or what lessons from past failures were learned.

### 1.3 The Three Ways

The model known as "The Three Ways" (Kim et al., 2013) encompasses the explanation associated with how improvement can be achieved. Flow: attainment of successful delivery is slow and out of sight; such a goal implies fast delivery involving visible flow encompassing a stages starting from commit to production. Feedback: feedback in the present time is received as email messages from angry customers; the goal is to make feedback fast and effective throughout the delivery. Continuous learning: improvement process is difficult to accomplish at the present time, as people are busy with problem solving; the aim is to make the process of improvement a part of every day work.

### 1.4 Lean waste in the current process

When applying Poppendieck and Poppendieck's (2003) distinctions of the seven wastes to TaskNest, it becomes clear that there are waiting (a developer waiting days for Priya to deploy), transferring of responsibility (from developer to "the person who knows the server"), late detection of defects (the defect reaches the customer since nothing has been tested against a production like environment), interrupted task (engineers being pulled into firefighting), partially done work (the features lingering in a branch for weeks), unnecessary processing (repeated manual server setup), and motion (engineers looking for files on the proper server). The best countermeasure in this case is the size of the batch; smaller batches are better (Humble and Farley, 2010).

### 1.5 Baseline DORA metrics

The initial step when dealing with the four DORA metrics (Forsgren, Humble and Kim, 2018) is to understand the issue in a measurable way:

Frequency of deploying: 2-4 times each month, done only when Priya is free (it's more of an informal practice).

Lead time from commit to the production: around 2-3 weeks, mostly due to waiting.

Change failure rate: estimated at 30-40% if a release initiates some incident (or at least a hotfix) according to estimates.

Time to restore service: 2 hours till 2 days depending on how fast Priya can be reached.

According to Forsgren, Humble and Kim (2018), top performers are able to release on demand with failure rate below 15%.

The justification is simple: fewer outages and faster delivery make funding phase easier to achieve and thus, a solid delivery practice has to be implemented.

## Part 2 - Version Control & Collaborative Workflow

### 2.1 Why version control discipline comes first

All aspects of this strategy are dependent on version control and it makes sense to start with that. This implies that all application code, infrastructure code, database migration scripts and pipeline

definitions will be stored in Git, meaning that only code in Git will ever be deployed into production. This addresses both issues the risk of 'only Priya knows' because information becomes code rather than knowledge and eliminates 'it works on my machine' scenario.

### 2.2 Branching strategy: trunk-based development

Trunk-based development of short lived feature branches is what I chose rather than GitFlow. GitFlow with its long-lived develop/release/hotfix branches is made for slow and scheduled releases. This is exactly the opposite of the SaaS team that deploys several times a week (Forsgren, Humble and Kim, 2018). The rules are simple :

The main is always in a deployable state and protected from direct pushes. Merging requires a green CI status.

Work is done in short lived branches for one or two days, not weeks.

Changes are made via pull requests with at least one reviewer checking correctness, clarity and test coverage. This way we spread knowledge.

We use conventional prefixes (feat: fix: chore:) for commits, so history is readable and changelogs are automated.

Every release is tagged (e.g. v1.4.0) allowing reproduction of any production state for the purposes of auditing or debugging.

Unfinished features are shipped via feature flags instead of existing in long branches, which helps to maintain the flow of work.

### 2.3 Evidence of practice

The Git log of the practice repository, as shown in Appendix A (under the corresponding artefacts file), depicts an orderly sequence of commits, a feature branch, the merging performed into the main branch, and the creation of a release with a tag v1.0.0.

## Part 3 - CI/CD Pipeline & Quality Strategy

### 3.1 Why CI: the integration problem

At present, the code produced by various developers is combined on a production server, which is considered one of the riskiest environments to come across any issues. Continuous Integration helps in tackling this situation because each code deployment on GitHub initiates a process during which code is built and tested within a short period of time (Fowler, 2006). The whole process is illustrated in Figure 1 and explained in detail in Appendix B.

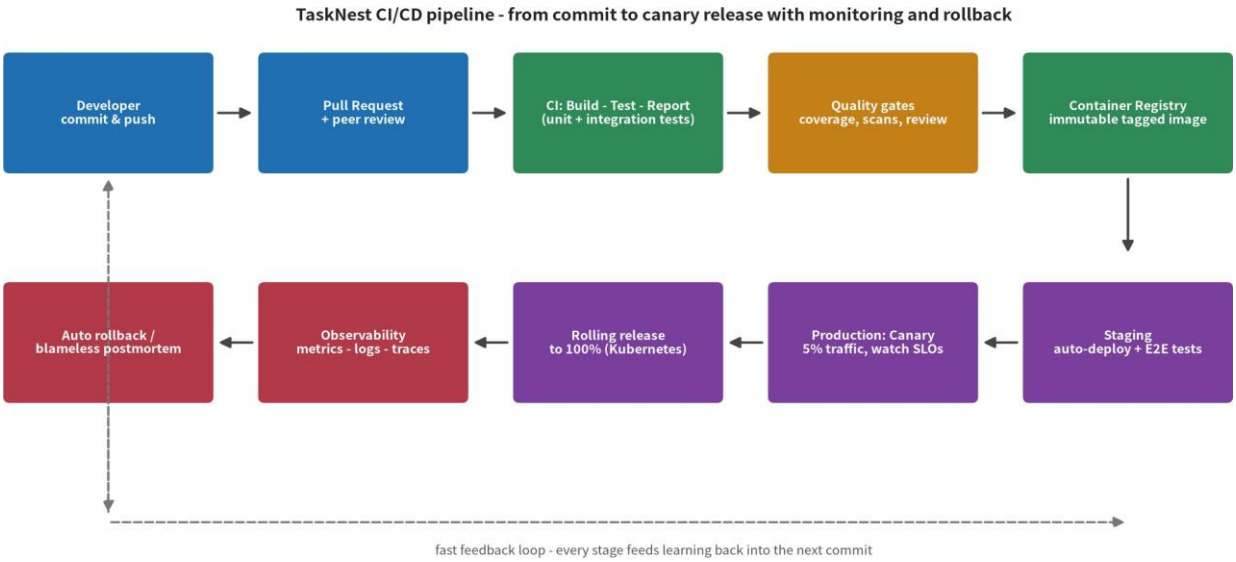


*Figure 1:* The CI/CD pipeline for TaskNest is delineated from commit to canary production deployment with monitoring and rollback.

### 3.2 The three CI stages: build, test, report

A pipeline run consists of three main phases. Build: install the necessary libraries and create the app as a Docker image. Thus, the same artifact is transformed from one phase to another. Test: run the automated tests. Report: report the results in the form of a status message on the pull request green means that it can be merged, and red means that the process has to stop and needs fixing. Fixing a broken main build is the number one priority of the team since it stops everything.

### 3.3 Testing strategy: the test pyramid

By adhering to the testing framework designed by Humble and Farley in 2010, TaskNest establishes tests that consist of a broad base and a thin top, making the company set up a total of:

Unit testing: basically about business logic, such as calculating invoices and verifying permissions. The testing should be limited to less than 120 minutes and cover about 70% of code paths.

Integration testing: fewer tests are done at this stage. They imply checking API endpoints and verifying that two correct systems can work together without bugs.

End-to-end testing: at this stage, only a few critical test cases remain, such as signing up, creating projects, inviting colleagues, and rolling payments.

The tests should be reliable so that the pipeline seems credible to customers. There are two principles: no shared state and deterministic tests, with flaky tests quarantined until bugs are removed and processes are repeated over time.

### 3.4 Quality gates

Quality gates, as automated measures, ensure the safety of the pipeline. Only in case all tests are passed; the coverage remains above the accepted level (which is 60% and increasing); the dependency and container scans don’t find any critical or high vulnerabilities and the Docker image

builds successfully; the pull request has been approved can the code be deployed in production.

### 3.5 Deployment and release strategy

In terms of a process called continuous delivery, this means that the main should always be in a state of readiness for release, with automation of builds from the main into a staging environment where end-to-end testing can be conducted (Humble and Farley, 2010).

Production is on Kubernetes, where the process is rolling deployment by default, meaning that new pods take over from the old one progressively, which minimizes the need for more infrastructure and is therefore suitable for small teams. In case of risky releases, a canary release is utilized; with a 5% of usage for a duration of under half an hour (potentially observing errors and latencies). I also checked the applicability of the blue-green deployment process (Fowler, 2010) but would not be able to afford sustaining two environments at this point of time. The way database changes are implemented is backwards-compatible (by expand and contract methods) and allows for coexistence of old code and new code during the deployment.

### 3.6 Rollback plan

Any deployment can be reversed in minutes: Kubernetes conserves old ReplicaSets, so rollback means 'kubectl rollout undo', which will lead to deploying the same immutable image instead of going back to the build. If the canary metrics (latencies and ratio of errors) are exceeded, the deployment stops automatically.

### 3.7 Automation choices and their risks

I automate tasks that are common, done repeatedly, and can lead to dangerous errors: build, test, deploy, and provision. Thus, I solve the major risks the pipeline creating a bottleneck (thus preventing it by caching dependencies and running tests in parallel, which completes the whole process in under 10 minutes); the issue of over automation of unreliable tests (which are kept small by design); and the risk of secrets leaking into logs (discussed in Part 5).

## Part 4 - Environments, Infrastructure & Containers

### 4.1 The goal: reproducible environments

TaskNest constructs its environments manually and humans are prone to committing mistakes. The solution exists in form of Infrastructure as a Code (IaC): environments are developed and maintained through code which is kept track of via Git, modified through pull requests and can be recreated anytime whenever needed (HashiCorp 2025).

### 4.2 Provisioning with Terraform

All resources in the cloud are specified using Terraform in a declarative manner, from the VPC and networking, to Kubernetes clusters and managed PostgreSQL database, object storage and IAM roles. Terraform places resources into reusable modules. The development, staging and production workspaces differ in the size of resources, so what happens in staging is similar to the production. The state is saved in S3 with lock functionality, as a single source of truth for all changes. Terraform plan runs as part of CI for every infrastructure pull request, which allows to review what changes would be made before applying them.

### 4.3 Containers with Docker

The application is packaged as a Docker image. Thanks to a multi stage Dockerfile, the final image only contains the runtime and the application code, making it lighter, quicker to deploy, and a smaller target for cyberattacks. Images are unchangeable and tagged with the SHA of the Git commit and the version of the software, which means that what works on CI is the same as what operates on production; the phrase "it works on my machine" is no longer applicable because the environment travels together with the code. On the local machine, one command is enough to run PostgreSQL and Redis using a Docker Compose file.

### 4.4 Orchestration with Kubernetes

Although it is simple to manage a single container, managing multiple reliably is a different matter altogether. That's where Kubernetes comes into play. TaskNest makes use of Deployments (which consist of defined replica counts and rolling updates), Services (which allow for seamless internal communication between the API, workers and database proxy), ConfigMaps and Secrets (which include configurations applied during runtime and does not require baking configuration into the image), Ingress (which is used for external HTTPS management), Horizontal Pod Autoscaler (which adjusts to workload while saving money), and liveness/readiness probes (which help with self healing of hung pods and avoiding delivery of requests to unready pods). Production occurs in several availability zones to ensure that failure of one zone does not take the service down.

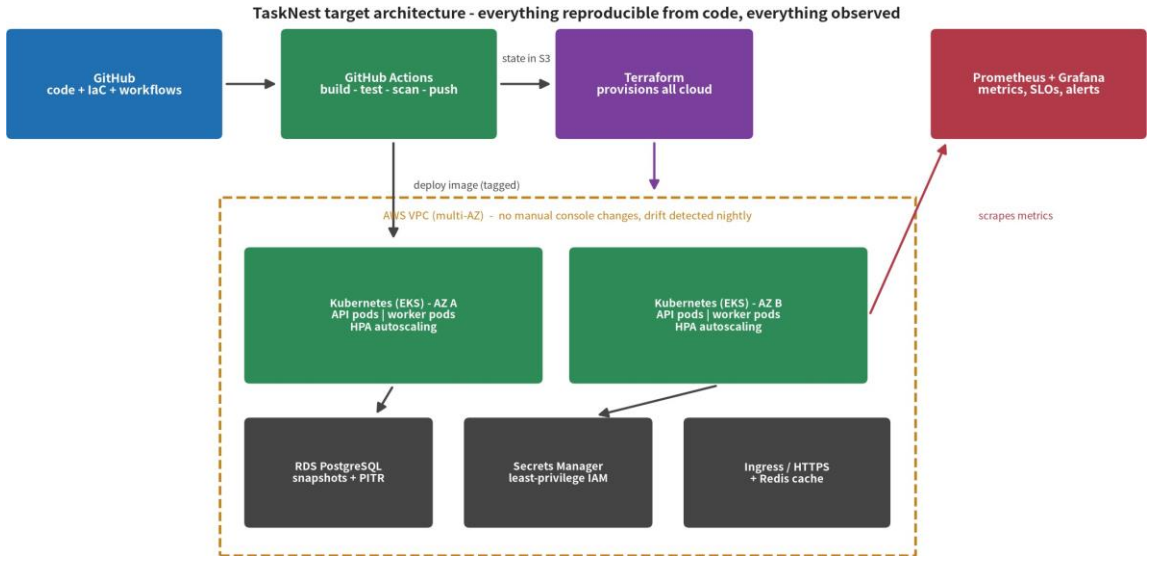


*Figure 2: Target environment architecture GitHub Actions builds and pushes the image; Terraform provisions the cloud; Kubernetes runs the application across availability zones; Prometheus/Grafana observe everything.*

## Part 5 - Observability, Security & Reliability

### 5.1 Observability: metrics, logs, traces

The present day TaskNest garners information regarding failures from clients. Observability refers to the idea that the system communicates news regarding performance through usage of three pillars (Beyer et al., 2016). Metrics: Prometheus collects four golden signals (saturation, errors, latency, traffic) from every service and presents them in Grafana dashboard for everyone to see. Logs: applications generate structured JSON (level, message, request ID) that is stored centrally (CloudWatch/Loki), so it is possible to track the request across many systems. Traces: through OpenTelemetry instrumentation, it is possible to follow a request through its API, worker, and database helping to answer the question 'why is this endpoint slow?' much faster than through

traditional methods.

### 5.2 SLOs and error budgets

The expression “be reliable” is vague so it has to be converted into figures, like SLOs altered into 99.9% availability (approximately 43 minutes of downtime a month) and the API latency on 95th percentile below 300 ms. The difference between perfection and SLO is known as error budget which determines the behavior: while the budget exists, the team works quickly; as soon as it is used up, the work on functionalities stops giving way to the work on reliability. Thus the issue of dev and ops confrontation is solved through a common metric on speed and stability (Beyer et al., 2016). The alerts are raised only in the case of user influencing problems and everything else turns into a ticket, since alerting on anything will make people ignore alerts.

### 5.3 DevSecOps: shifting security left

Security participates in the delivery process rather than performing an audit afterwards. AWS Secrets Manager stores the secrets outside of the source code and configuration files and they are injected at runtime using IAM roles with the least privileges nothing sensitive in Git. Dependency scanner (Dependabot) and static analyzer (CodeQL) are run on each pull request; images are checked using Trivy in the pipeline, and the significant vulnerability does not pass the quality gate. Branch protection and required reviews are policy as code: the rules work themselves. In addition, during due diligence for the fundraising round, this becomes proof of compliance in the form of the audit trail (who, what, when, who approved it, how was it tested) created automatically by Git and the pipeline.

### 5.4 Incident response and post-incident learning

While incidents are inevitable, it is the response process that makes all the difference. There is a severity scale (SEV1 = complete failure to SEV3 = minor issue) that determines the priority of the response. Every SEV1/2 incident is assigned an incident commander who coordinates activities and another person to mitigate the incident; both these persons work together and keep a record of the timeline to avoid conflicting orders.

Less focus on the cause, more focus on the mitigation process: when the customers are involved, rollback is more important than identifying the reasons. Following the outcome of each SEV1/2 incident, a no blame assessment is done within 48 hours because when people are afraid of getting blamed, they hide important information; and when information is kept hidden, incidents are bound to repeat themselves.

### 5.5 Disaster recovery

The objectives for disaster recovery are coordinated with the business, and include an RTO of 1 hour (i.e., downtime should not exceed 1 hour) and an RPO of 15 minutes (i.e., maximum data loss). In order to achieve the objectives, it is required to automate database snapshots creation every 15 minutes with the help of the point in time recovery feature; the entire infrastructure needs to be rebuilt in another account or region using terraform; application state is stored outside the pods making them disposable; and finally, what everyone forgets about is performing quarterly restore rehearsal because the backup that has not been tested is no more than a hope.

### 5.6 Trade-offs

These measures have financial implications, and it would not be truthful to act as if otherwise. The use of canaries and error budgets can slow the feature flow; however, the downtime experienced remains within tolerance levels, scanning may take some extra time, but it blocks all known

vulnerabilities and using multiple availability zones costs more than using one zone, but is capable of surviving zone failure. The usage of funds should be focused where real risk is; payment processing receives maximum safety, whereas internal administrative pages are not designed for high levels of security.

# Part 6 - Cloud, Cost & Leading the Change

## 6.1 FinOps: making the bill somebody's job

It is essential that costs associated with cloud services be monitored on an ongoing basis rather than viewed as merely a one-time ‘clean-up’ issue. The following measures should be taken: all resources must be tagged according to team, environment and owner; budget alerts for owners should be sent out at 80 percent and 100 percent usage; resource consumption for the development environment should be scheduled to be scaled down when not being used (outside working hours); capacity should be adjusted dynamically via the autoscaler as per advice; budget reviews should be performed on a monthly basis.

## 6.2 The change roadmap

Accomplishing everything at once will be unsuccessful, so the delivery will be phased over approximately six months, with measurable value in each phase: Phase 1 (months 1 to 2) foundations: Git for everything, trunk-based development with PR reviews, CI pipeline on every single push. Quick win: developers will not have to wait for Priya to incorporate the code anymore. The second phase (months 3 to 4) environments: Terraform for pre-production and production, Docker containers, automatic deployments to the staging environment, as well as Compose for local development. A quick win here: newly employed people can be productive on the same day they were hired. Phase 3 (months 5 to 6) operations: deployment of Kubernetes in production, rolling/canary releases with rehearsed rollback strategy, Prometheus/Grafana for monitoring, SLOs, security gates, and the first after-incident blameless review. As a result, delivery becomes a routine process - exactly what is required for the funding round.

Phase 2 (Months 3-4) Environment setup: we will implement Terraform for production and preproduction environments, utilize Docker packaging and automate the deployment process to preproduction, and use docker compose for local development. Advantage: new employees require only a day to adapt.

Phase 3 (Months 5-6) The set of operations: we will implement Kubernetes for production, apply rolling processes like rolling or canary releases and the rollback rehearsal, use Prometheus and Grafana, define SLOs, create security gates during scanning, and begin learning from the process without blaming anyone. The most significant advantage is that the process transitions into routine, which is very useful to justify the funding request.

This change is the biggest challenge: from the belief that “Priya is responsible for the service” to “the team is responsible for the product you create it and then you operate it.”

## 6.3 Measuring success and conclusion

The effectiveness of the strategy is evaluated according to the Part 1 set of indicators, which are measured weekly and include: the deployment times (increased from 2x/month to 1/week and even to real time deployments), leading time (decreased from weeks to less than a day), failure change rate (dropped from 35% to less than 15%), and recovery timeline (reduced from days to one

hour). In addition to these, the cloud costs per user either remain stable or decrease, and the drift detection mechanisms detect zero secrets or manual changes. Indicators above help assess whether the strategy delivers the desired results and, if not, the reason for it, which aligns with the core ideas and principles of DevOps.

Stating that no extraordinary engineering or lucky releases are needed in the case of TaskNest, it can be concluded that the company is looking for a standardized process of service delivery.

## References

**Beyer, B., Jones, C., Petoff, J. and Murphy, N.R. (eds.) (2016)** *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA: O'Reilly Media. O'Reilly – Site Reliability Engineering

**Forsgren, N., Humble, J. and Kim, G. (2018)** *Accelerate: The Science of Lean Software and DevOps*. Portland, OR: IT Revolution. IT Revolution – Accelerate

**Fowler, M. (2006)** 'Continuous Integration'. Available at: Martin Fowler – Continuous Integration
(Accessed: 26 August 2026).

**Fowler, M. (2010)** 'Blue Green Deployment'. Available at: Martin Fowler – Blue Green Deployment
(Accessed: 26 August 2026).

Terraform Documentation (Accessed: 26 August 2026).

**Kim, G., Behr, K. and Spafford, G. (2013)** *The Phoenix Project: A Novel About IT, DevOps, and Helping Your Business Win*. Portland, OR: IT Revolution. IT Revolution – The Phoenix Project

**Kim, G., Humble, J., Debois, P. and Willis, J. (2016)** *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. Portland, OR: IT Revolution.
IT Revolution – The DevOps Handbook

**Kubernetes (2025)** *Kubernetes documentation*. Available at: Kubernetes Documentation
(Accessed: 26 August 2026).

**Nygard, M.T. (2007)** *Release It!: Design and Deploy Production-Ready Software*. Raleigh, NC: Pragmatic Bookshelf. O'Reilly – Release It!

**Poppendieck, M. and Poppendieck, T. (2003)** *Lean Software Development: An Agile Toolkit*. Boston, MA: Addison-Wesley.Google Books – Lean Software Development

**DORA / Google Cloud (2023)** *Accelerate State of DevOps Report*. Available at:
DORA – Accelerate State of DevOps Report 2023 (Accessed: 26 August 2026).